DTU Compute
Department of Applied Mathematics and Computer Science

Advanced Programming (02324), Spring 2026

Carlos E. Budde and Ekkart Kindler

Assignment 3: Generic Sorted Lists & Java Collections

Assignment 3 revisits and extends all you have learnt for Stacks, by challenging you to develop implementations for an interface of another main abstract data type—Lists:

- *in **Assignment 3a** (Friday, February 13) you will use arrays to implement the essential functionality of a generic `List<E>`, as defined in the interface provided to you;*
- *in **Assignment 3b** (Tuesday, February 17) you will implement an algorithm to sort the elements inside the list, plus the data type `SortedList<E>` which is not quite the same;*
- *in **Assignment 3c** (Friday, February 20) you will extend the GUI that operates with these implementations, using Java Collections to add useful functionality.*

These assignments are incremental, meaning that they build on each other. You start by downloading the IntelliJ/Maven project `assignment3.v1.1.0.zip` from DTU Learn. Assignment 3a begins from those base files; once you have finished and presented your solution, Assignment 3b continues from that solution; ditto Assignment 3c.

The final complete solution to Assignment 3 will be the project that your group submits for Part 1 (out of 4) of the course—see Submission of projects in Exam and Submissions (DTU Learn).

Assignment 3b: Sorting Lists and Sorted Lists

In Assignment 3b, you must implement the `sort(Comparator<? super E> c)` method of your previous code for `ArrayList<E>`, to be able to sort a generic `List<E>`. You will also provide an implementation for `SortedList<E>` by writing code for `SortedArrayList<E>`. In detail:

0. **Continue** where you left off: Before you start with this assignment, a member of your group must have presented and have approved Assignment 3a.
   - Your work for Assignment 3b starts from the code for your solution of Assignment 3a.
1. **Read** the documentation provided (as JavaDoc comments) for the the `sort(…)` methods in `ArrayList<E>` and `BubbbleSort<E>`, to understand what is expected of each method in terms of both normal functionality, and exceptions to be thrown/caught.
2. **Implement** those methods as taught in the lectures of Friday, February 13, and Tuesday, February 17. All the methods that must be implemented are marked with a comment `// TODO ... (Assignment 3b)`.
   - Test your implementation incrementally by hand, running `uses/PersonApp.java` and using the `Sort` button of the GUI for the `(unsorted) List` implementation.
3. **Read** some more: when you are done with the implementation to sort a “normal” (aka “unsorted”) list, move on to implement the `SortedList<E>` generic data type. For that, start by reading the documentation provided in `types/SortedList.java`, to understand what is expected of each method (normal functionality + exceptions).

4. **Implement** `SortedArrayList<E>` in a manner that complies to that documentation. All methods to implement are commented with `// TODO ... (Assignment 3b)`.
5. **Test** your solutions for the tasks above, which can be considered sufficient only after all tests in `TestSortedArrayList` and `TestForAllLists` are passing successfully.
6. **Present** your solution to the teachers or teaching assistants during the tutorial sessions on Tuesdays or Fridays, at latest on Friday, February 20.
   - If you can, you're strongly advised to present on Tuesday, February 17, since the final assignment depends on this solution.
7. If you should have time left ***after presenting Assignment 3b***, you can extend the tests cases in `TestArrayLists` and `TestSortedArrayList`. For that, look for the comments `// TODO there could be some more tests concerning...` therein, that provide some minimal hints on what methods are mostly in need of further testing.
   - Moreover, if you have implemented helper functions to solve the assignment (that's typically a sign of good coding), add tests for those functions as well. As a rule of thumb, consider that no function should be left untested ("full test coverage").

References and further reading

[02324 f26 L04, L05] Carlos E. Budde and Ekkart Kindler: Lecture Notes for the course Advanced programming. Spring 2026 (DTU Learn).

[Eck 2022] David J. Eck: Introduction to Programming Using Java. Version 9.0, JavaFX Edition, May, 2022. Sections 10.1.6 and 10.2.2